Collective Communication for Modern GPU Training in gem5 Garnet

A measurement-led study of multicast, express links, and tensor streaming

Chunyu Liu and Boyan Pu

August 28, 2026

Abstract

Distributed GPU training repeatedly pauses computation for communication. We ask which parts of that communication should be exposed to a Garnet Mesh network rather than lowered into unrelated unicast packets. The study has three mechanisms—router-level tree multicast, physical express-link bypass, and multi-flit in-network all-reduce—and uses an executed eight-H100 collective trace as an integrated case study of those mechanisms. The implementation passes all seven correctness gates, 12 trace-tool tests, 162 multicast pairs, 6,144 bypass pairs, and the 30-request broadcast and 15-request tensor replays. Tree multicast cuts internal-link flits by 39.51% on average (26.19%, 39.22%, and 53.13% for 4, 8, and 16 destinations), but its atomic fanout path has ten throughput regressions. In the broad static-oracle topology sweep, distance-scaled diagonal links reach 1.629× latency speedup, while stride links average 0.981×. Finally, preserving 15 multi-flit tensor requests instead of serializing 6,720 scalar lanes reduces the scaled replay window from 80.638 to 17.055 million ticks (4.728×). These are cycle-level network results, not H100 hardware or end-to-end model-training speedups.

1 Introduction

Modern training gives the network a repeated synchronization contract. Each GPU computes locally, exchanges a gradient or activation, and waits until the collective result is available before the next step can consume it. The same contract produces three distinct network opportunities:

1. **Replication:** a broadcast can share a common route prefix and replicate only at branch Routers.
2. **Path length:** a unicast packet can skip intermediate Routers when a physical express link is worth its wire cost and congestion.
3. **Representation:** an all-reduce tensor can remain a streaming multi-flit request instead of becoming thousands of serialized scalar operations.

This report focuses on Lab 4 Topic 3 (Microarchitecture), which requires both Bypass and Multicast. Tensor all-reduce and replay are an additional Topic 4 extension that makes the AI-training setting concrete. The implementation has three corresponding parts:

- **Router-level tree multicast** shares common route prefixes and replicates a single destination-bitmap packet at branch Routers.
- **Physical express-link bypass** adds shortcut links and admits a source express hop when the offline route oracle proves a strict path shortening and the selected topology passes the dependency audit.
- **Tensor all-reduce replay** keeps a measured collective as a multi-flit request, reduces lanes in Routers, and broadcasts the result.

The primary performance experiment is the static-oracle bypass screen across topologies and wire models. The multicast matrix and multicast–bypass interaction are supporting mechanism studies; the H100 replay is a concrete trace case study rather than a replacement for the factorial evaluation.

The main conclusions are intentionally asymmetric. Multicast has a robust traffic benefit but a workload-dependent throughput benefit. Bypass is not a general hop-count win: it is useful on structured or loaded traffic only when admission avoids harmful shortcuts. Tensor streaming changes the amount of logical work observed by the replay without changing the number of rank contributions or Router flits. Keeping these claims separate prevents a traffic reduction, a latency ratio, and a trace-window ratio from being read as the same metric.

2 Architecture and implementation

Figure 1 makes this motivation concrete. A single training step alternates between local GPU work and a synchronization phase: each rank computes a gradient shard, the ranks exchange or reduce that shard, and only then can the next step safely consume the result. The figure maps that application-level story to the Mesh NoC and Router datapath evaluated in this report.

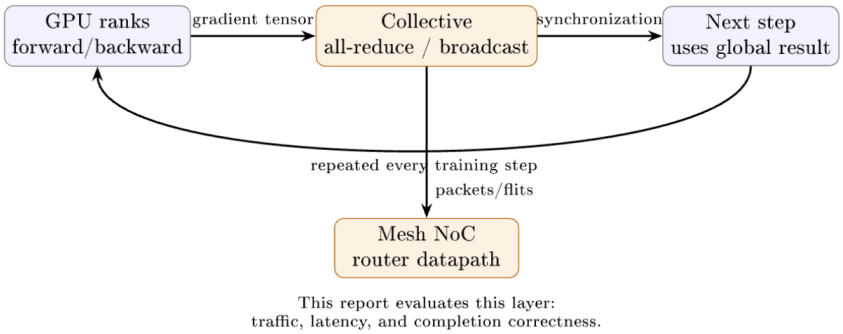

Figure 1: From a modern multi-GPU training step to the network mechanisms evaluated in this report. The communication phase is on the critical path when computation cannot proceed until the collective completes.

2.1 Variants and baselines

The released static G9 snapshot uses gem5 v23.0.0.1 (revision `77fcf26d57`) and deterministic XY routing on a Garnet Mesh. A reported ratio always compares the same topology, traffic, packet sequence, and seed; the comparison contract is summarized in Table 1:

Table 1: The comparison contract for the main multicast and bypass studies.

Study	New mechanism	Baseline and invariant
Multicast	One destination-bitmap packet, replicated at branch Routers	4, 8, or 16 independent packets on the identical Mesh; one complete packet must arrive at every destination.
Bypass	Mesh with physical express links and a source routing oracle	The same Mesh with no express links; packet injection and XY suffix are unchanged.

For a paired latency L, internal-link flit count F, and throughput Q, we use

$$S_L = L_{\text{base}}/L_{\text{new}}, \qquad R_F = 1 - F_{\text{new}}/F_{\text{base}}, \qquad I_Q = Q_{\text{new}}/Q_{\text{base}} - 1.$$

Thus $S_L > 1$ is faster, $R_F > 0$ is less link traffic, and $I_Q < 0$ is a throughput regression. Arithmetic means are used where the distribution is interpretable; bypass latency aggregates use geometric means because a few near-saturation ratios have a very long positive tail. Minima, maxima, and negative cases remain in the released CSV files.

2.2 Tree multicast

The baseline Network Interface injects one physical packet per destination. The multicast path injects one packet carrying a 64-bit destination bitmap. Each Router removes destinations already served, computes the required output branches, and makes a local copy only where selected. Head, body, and tail state is retained per branch. Fanout is atomic: a flit advances only when all selected output VCs have credit. This preserves exact multi-flit delivery, but it also couples a fast branch to a blocked branch—an important interpretation of the throughput results below. The bitmap bounds one collective domain to 64 Routers.

2.3 Static express-link bypass

`Mesh_Bypass` appends bidirectional diagonal or stride links to the ordinary Mesh. An offline oracle enumerates every source/destination pair and keeps at most one source express hop when it strictly reduces Manhattan distance, followed by a deterministic XY suffix. The oracle also builds the channel-dependency graph for all pairs and rejects any placement whose graph is cyclic. The runtime route is deterministic: the source follows the oracle's selected express first hop, if any, and every remaining hop follows XY. The main implementation has no per-hop adaptive choice, so runtime behavior matches the audited route table exactly.

- **Optimistic:** every express link costs one cycle, an upper bound on the value of shortening a path.
- **Distance-scaled:** latency and serialization scale with the Manhattan span, a more conservative proxy for a physical wire.

The main G9 runs use the static oracle. For each source/destination pair, the oracle selects at most one express hop when it strictly reduces the remaining Manhattan hop count; otherwise the packet starts with XY. Ordered Vnets retain their ordinary static route. An optional adaptive selector remains available behind an explicit flag, but it is not used for the headline results here.

1. The source consults the oracle's first-hop entry for the destination.
2. If the entry names an express output, the packet takes that physical link; otherwise it takes the ordinary XY first hop.
3. After the first hop, routing follows the deterministic XY suffix.
4. The oracle's channel-dependency DAG supplies the deadlock-safety argument; no runtime route choice can introduce an unaudited dependency.

The G9 screen uses data VNet 2 (four buffers per VC). The express-link cost is either optimistic (one cycle) or distance-scaled by the link's Manhattan span; the latter is the hardware-relevant model used for the main comparison.

3 Evaluation results

3.1 Multicast performance

Across 162 paired cases, tree multicast reduces internal-link flits by 39.51% on average (median 41.18%, range 16.67–53.13%). The effect is monotone in fanout: 26.19% for four destinations, 39.22% for eight, and 53.13% for sixteen. This is the cleanest evidence for the mechanism because both variants use the same Mesh and the metric counts only physical link flits. Figure 2 summarizes the reduction by fanout.

Latency also improves in this matrix: mean speedup is 4.790× (median 3.588×, range 1.200–22.512×), and all 162 paired values exceed one. That result should not be interpreted as a pure consequence of shared prefixes: the multicast implementation has a dedicated Router fast path and therefore does not traverse exactly the same switch-allocation/crossbar path as unicast. We report the latency as an implemented-system result and use link flits as the structural claim.

Throughput is the necessary counterexample. The arithmetic mean change is +190.89%, but the distribution is highly skewed by full-group traffic; ten of 162 cases regress, all at four destinations, with a worst case of −18.24%. The paired distribution is shown in Figure 3. Atomic fanout explains how both statements can be true: the tree does less total link work, yet a single blocked branch can hold up all copies.

The workload breakdown makes the dependency concrete. At 4, 8, and 16 destinations, mean throughput changes are +34.38%, +154.87%, and +383.42%, but the four-destination group contains all ten regressions. By packet size, the mean changes are +261.65% (1 flit), +169.45% (4 flits), and +141.57% (16 flits); the negative cases concentrate in the longer, loaded four-way requests. Figure 4 shows the same pattern without hiding the min–max spread, while Figure 5 relates traffic reduction to throughput on a case-by-case basis.

Multicast takeaway. Tree replication reliably removes duplicate physical work, and the saving grows with fanout. Throughput is not guaranteed to follow because atomic fanout and the dedicated fast path introduce their own scheduling effects.

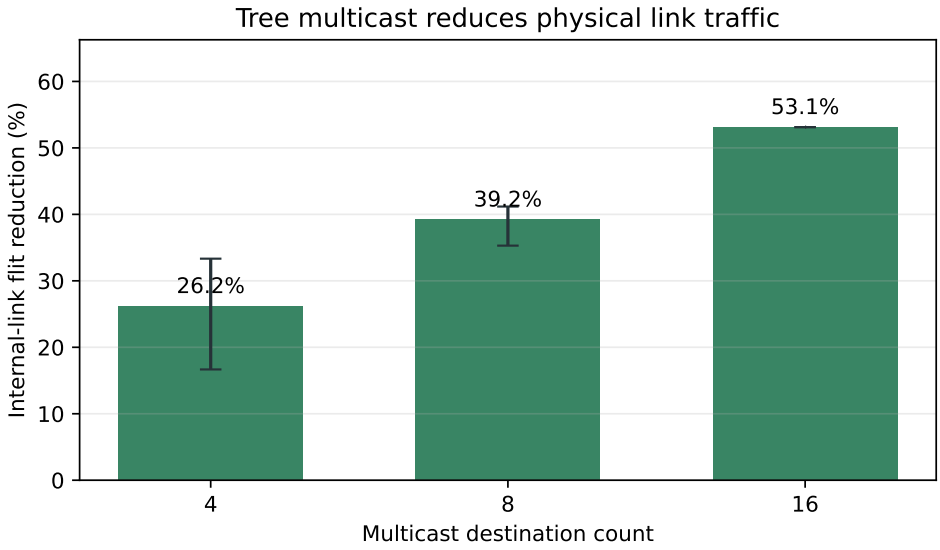

Figure 2: Internal-link flit reduction from tree replication. Bars are means over 54 paired cases per fanout; whiskers span observed minima and maxima.

3.2 Primary experiment: static-oracle bypass

The bypass sweep uses four standard synthetic traffic patterns. In `uniform_random`, each source chooses a destination uniformly at random. In `transpose`, a source at mesh coordinate (x, y) sends to (y, x), creating a structured permutation. In `bit_complement`, it sends to the mirrored coordinate $(W - 1 - x, H - 1 - y)$, pairing opposite locations in the mesh. In `hotspot`, 50% of packets target one fixed Router (Router $N/2$ in an N-node mesh) and the rest use uniform random destinations. These are controlled stress patterns for route structure and contention; they are not claims about four separate application workloads.

The G9 sweep is the primary bypass result. It compares the same synthetic workloads and seeds on Mesh XY and on each physical express-link design, with the static oracle selecting at most one source express hop. The numbers in Table 2 are the arithmetic means over all 1,536 paired seed cases per design. The static oracle is intentionally an upper-bound policy: it proves route legality and path shortening offline, but does not claim that every shortcut is beneficial under every offered load. Stride still removes more Router traversals, but its physical wire cost is higher and its latency benefit is smaller than diagonal links. Optimistic wire timing remains an upper-bound sensitivity point rather than physical evidence.

Table 2: G9 topology and wire-model screen over 1,536 paired seed cases per design.

Design	Latency speedup	Throughput change	Traversal reduction
Diagonal, distance-scaled	1.629×	+6.41%	−2.82%
Diagonal, optimistic	1.723×	+10.61%	−7.03%
Stride, distance-scaled	0.981×	−1.94%	+15.50%
Stride, optimistic	1.103×	+3.68%	+10.10%

The cost records explain the design choice. A 4×4 diagonal placement adds 9 undirected links (wire proxy 18, maximum radix 8), while stride adds 16 (wire proxy 32, maximum radix 6); on 8×8 the corresponding link counts are 49 and 96. Hop reduction alone is therefore not a sufficient argument for a physical topology.

Figure 6 summarizes the four static-oracle designs. Diagonal links provide the strongest latency result, while stride links remove more Router traversals but incur a larger physical-link budget. The optimistic wire model is a sensitivity bound; distance-scaled latency is the hardware- relevant comparison.

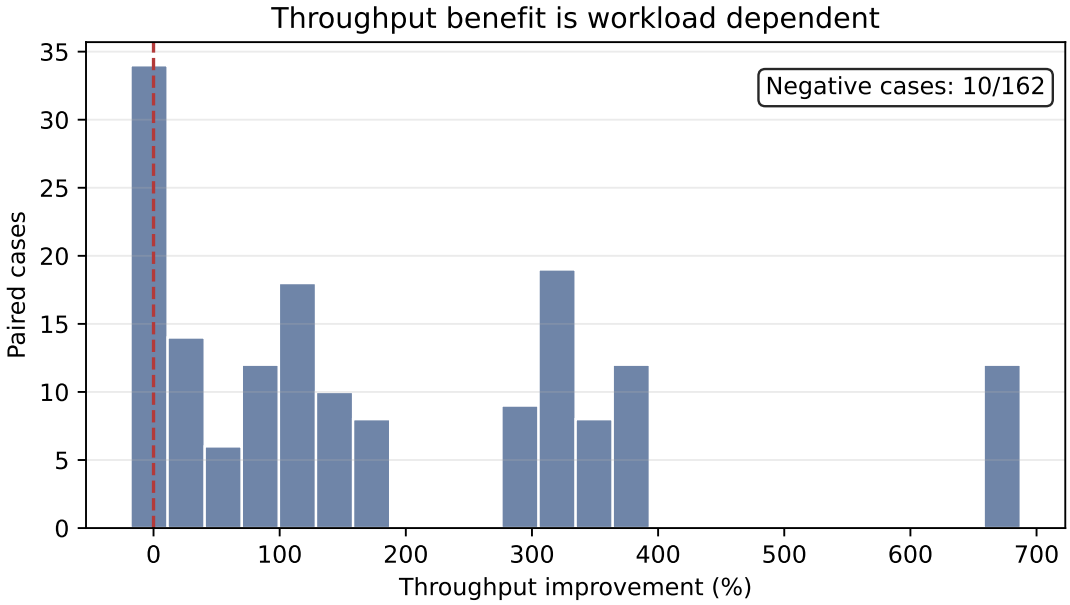

Figure 3: Paired multicast throughput changes. The long positive tail comes from high-fanout traffic; the ten values left of zero are all four-destination cases.

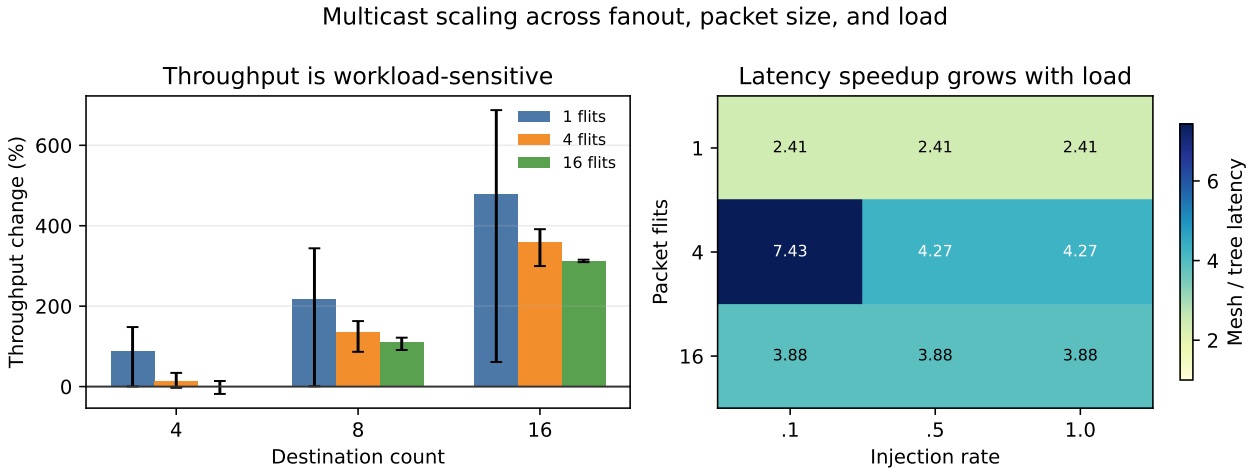

Figure 4: Multicast sensitivity to fanout, packet size, and offered load. The left panel reports mean throughput change with observed extrema; the right panel reports geometric-mean latency speedup across background conditions.

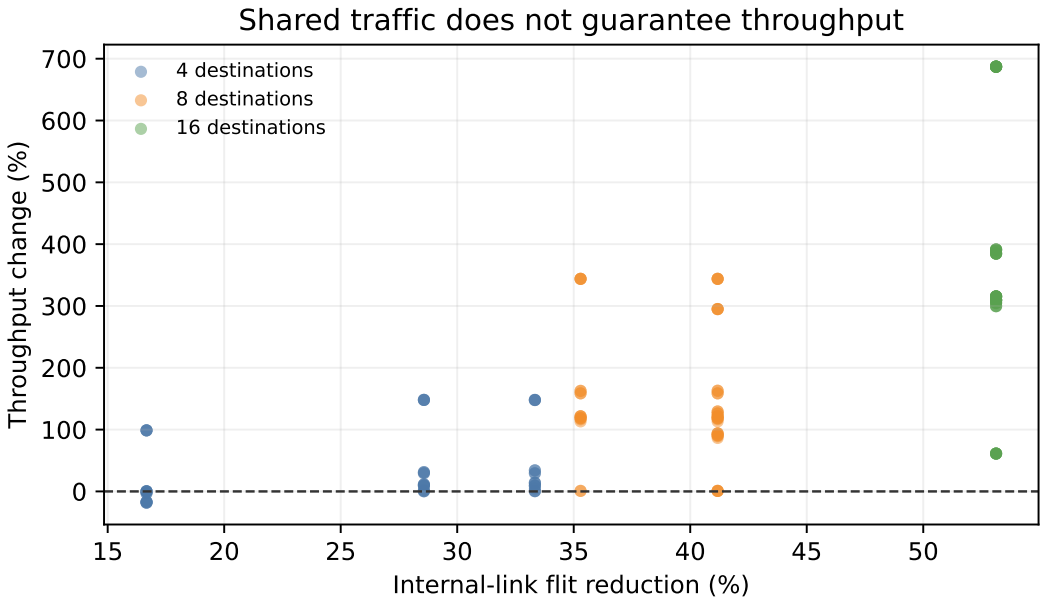

Figure 5: Traffic reduction and throughput are related but not equivalent. Each point is one paired case; color denotes destination count and the dashed line marks unchanged throughput.

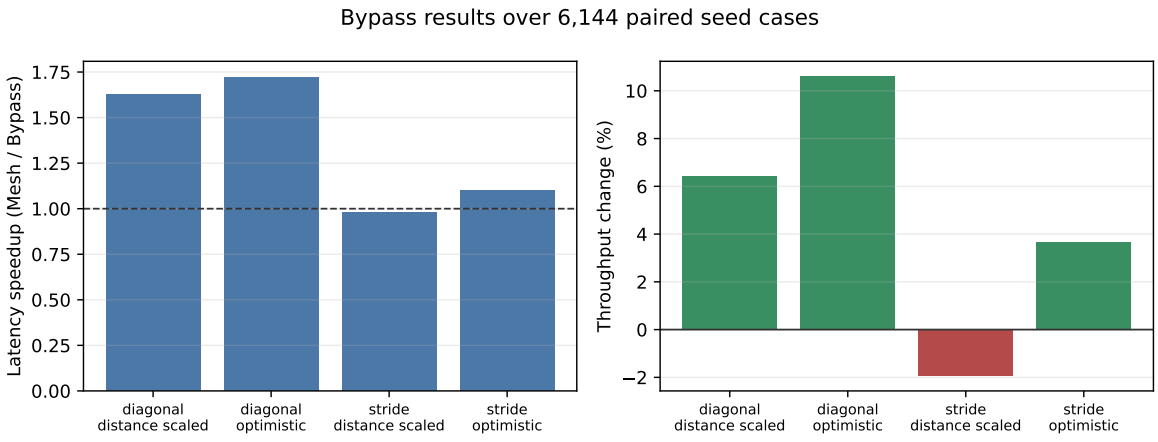

Figure 6: Static-oracle bypass results over 1,536 paired seed cases per design. The left panel reports Mesh/bypass latency speedup and the right panel reports throughput change.

Multicast–bypass interaction. G10 contains 416 paired cases and all 624 raw runs pass. Under distance-scaled links, bypass averages 0.965× for replicated unicast and 0.988× for tree multicast; the optimistic values are 1.029× and 1.022×. Tree multicast uses 85,000 express-link flits across the paired runs versus 357,000 for unicast. The interaction therefore does not justify forcing every multicast branch through a shortcut: preserving the ordinary tree is often the lower-risk choice.

Bypass takeaway. Express links are useful when route structure makes a saved hop valuable, but distance-scaled wires and extra radix impose real costs. The static oracle is therefore best interpreted as a performance upper bound; the workload sweep shows where that upper bound survives realistic wire timing.

4 H100 trace case study

This section is intentionally separate from the factorial multicast and bypass evaluation. It documents the trace-specific lowering, validation contract, and replay results; none of its scaled tick counts are folded into the main-study aggregates.

4.1 Trace lowering and implementation

The source is an executed eight-NVIDIA-H100 80GB HBM3 NCCL collective microbenchmark (PyTorch 2.8.0+cu128, CUDA 12.8, NCCL 2.27.3), not a full model-training trace. The replay compiler selects the 15 measurement-phase broadcasts and 15 all-reduces, represents communication with 16-byte flits, and scales payload bytes and relative release times by 1/1024. This keeps the event ordering reproducible while changing absolute volume and timing.

For broadcast, each selected source event is lowered to variable-length tree-multicast packets. For all-reduce, the scalar replay creates independent serialized lanes; each lane converges rank values in the Routers and then travels back down the convergence tree. The tensor replay keeps each event as one multi-flit logical request, tracks a lane ID per flit, and performs the same Router-side reduction and tree broadcast without scalarizing the request. The scalar and tensor paths are separate collective implementations from the bitmap-based `tree_multicast` path, although all three exploit shared tree prefixes.

All three replay runners use the same static-oracle Mesh_Bypass arm as the main G9 study: diagonal links with a four-link budget and distance-scaled wire latency. Mesh XY is the matched no-express baseline. In this H100 placement the oracle selects no express hop, so the bypass comparison is deliberately neutral.

4.2 Replay experiments and results

The replay uses one 2×4 topology for both Mesh XY and Mesh Bypass. It contains a broadcast replay plus two lowerings of the same 15 all-reduce events; the 4.728× comparison below is therefore a representation comparison, not a fourth workload.

This trace is a concrete composition of the project mechanisms, rather than a separate H100 hardware study. The all-reduce first converges rank values in the Routers and then disseminates the result with a tree broadcast, applying the same shared-prefix principle as multicast (the all-reduce and `tree_multicast` paths are distinct implementations). The Mesh Bypass run supplies the paired topology comparison. In this particular placement and routing policy the collective oracle selects no express hop, so the trace demonstrates mechanism compatibility and correctness but does not claim a bypass speedup.

The broadcast replay expands 15 source events into 30 tree-multicast requests; both designs complete 6,720 source flits and 210 destination deliveries in a 17,047,500-tick window. The static oracle selects zero express hops, so this is a controlled neutral bypass result rather than evidence of a bypass speedup.

Table 3 records the three replay views side by side. The Mesh XY and static Mesh Bypass columns are identical for every entry because the H100 oracle selects no express hops; this equality is itself a useful check that adding physical links does not perturb the trace when no shortest- path shortcut exists.

For all-reduce, both physical designs complete 15 logical tensor requests, 53,760 rank contributions, 53,760 exact lane reductions, 53,760 deliveries, and 147,840 Router flits. The tensor window is 17,054,500 ticks; p50/p95/p99 logical-request latencies are 647,500/2,567,500/2,567,500 ticks. These percentiles describe multi-flit requests and must not be compared directly to the scalar-lane p95 of 10,000 ticks. Figure 7 visualizes the only sound cross-lowering comparison: the end-to-end measurement window.

Table 3: Scaled H100 replay summary (both topology arms).

Replay view	Logical requests	Source flits	Window (ticks)	p95 (ticks)
Broadcast	30	6,720	17,047,500	643,500
Scalar all-reduce lanes	6,720	53,760	80,638,000	10,000
Tensor all-reduce	15	53,760	17,054,500	2,567,500

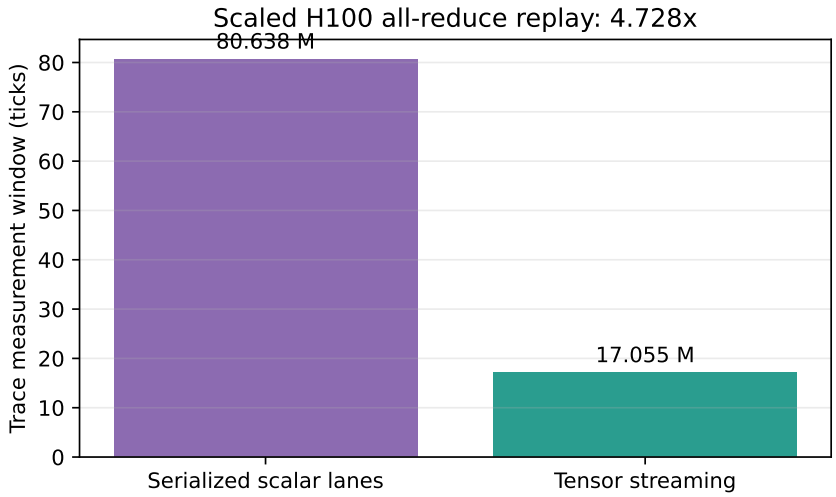

Figure 7: The same scaled H100 all-reduce trace under two logical lowerings. Tensor streaming keeps 15 requests intact; scalar replay serializes 6,720 lanes. Bars show the end-to-end measurement window, the only sound cross-lowering comparison.

Scalar lowering records an 80,638,000-tick window, so tensor streaming is 4.728× shorter on both Mesh XY and Mesh Bypass. Source and Router flit counts are identical, which isolates the measured gain to request representation and completion scheduling, not to fewer rank contributions or less network traffic. It is a replay-level result: the model does not claim that a real H100 would obtain a 4.728× training-step speedup.

Trace-replay takeaway. Keeping a tensor as a streaming collective removes scalar serialization in this network model while preserving exact arithmetic and delivery invariants. The benefit is visible in the total trace window, not in incomparable per-request tail percentiles.

5 Limitations and interpretation boundary

The evidence supports the mechanisms within the tested model, with the following boundaries:

- Multicast uses one 4×4 source placement (Router 5), so corner, edge, and center sources are not separated. Its fast path differs from the full unicast allocator, and atomic fanout can serialize branches.
- Bypass wire cost is a Garnet latency/serialization/static-cost proxy; it does not model area, power, repeaters, or timing closure. The the static oracle uses at most one source express hop and the suffix is deterministic XY.
- G9 uses synthetic unicast traffic on data VNet 2. The distance-scaled screen is the hardware-relevant comparison; optimistic timing is only a sensitivity bound.
- The H100 input is an executed collective microbenchmark, not a full model trace, and its 1/1024 scaling changes absolute time and volume. Broadcast and tensor replay selected zero express hops on the shared topology.
- Tensor accumulation has unbounded software state and no modeled arithmetic pipeline or finite table. Chunk boundaries are validated by the contract but are not a runtime accumulator dimension.
- None of the results measures NVLink/NVSwitch hardware or end-to-end model throughput. They characterize cycle-level Garnet mechanisms.

6 Division of labor

Chunyu Liu authored multicast and bypass implementation, gates, trace capture, optimization, and the associated analysis. Boyan Pu authored the tensor all-reduce datapath, replay contract, validator, backpressure gates, and tensor statistics. Both authors integrated branches, reviewed regressions, reran the final quantitative snapshot, and verified this report.

7 Reproducibility and artifact trail

Compact accepted snapshots are in `report/data/`; generated vector and preview figures are in `report/figures/`; raw outputs are git-ignored under `report/raw-results/head-77fcf26d/`. The source trace and replay hashes, software versions, scaling semantics, and artifact roots are recorded in `report/data/trace_replay_provenance.txt`. The compact-data builder rejects wrong run counts, non-finite values, failed gates, inconsistent VNet or routing metadata, incomplete multicast pairs, and missing designs before writing summary tables or figures.

The principal commands are:

```
python3 tests/gem5/lab4/run_lab4_full_gate.py \\
  --gem5 build/Garnet_standalone/gem5.opt --jobs 64
python3 tests/gem5/lab4/run_multicast_performance.py \\
  --gem5 build/Garnet_standalone/gem5.opt --jobs 32
python3 tests/gem5/lab4/run_bypass_performance.py \\
  --gem5 build/Garnet_standalone/gem5.opt --inj-vnet 2 \\
  --sizes 4 8 \\
  --traffic uniform_random transpose bit_complement hotspot \\
  --packet-flits 1 4 16 64 \\
  --offered-loads 0.01 0.025 0.05 0.1 0.2 0.3 0.5 0.7 \\
    0.9 1.0 1.25 1.5 1.75 2.0 2.5 3.0 \\
  --families diagonal stride \\
  --wire-models optimistic distance_scaled \\
  --jobs 64
python3 tests/gem5/lab4/run_multicast_bypass_matrix.py \\
  --gem5 build/Garnet_standalone/gem5.opt --jobs 64
python3 tests/gem5/lab4/run_trace_replay.py \\
  --gem5 build/Garnet_standalone/gem5.opt \\
```

```
  --source-trace traces/h100_8gpu_collectives.json \\
  --replay traces/h100_8gpu_broadcast_replay.json \\
  --sim-cycles 100000000
python3 tests/gem5/lab4/run_allreduce_trace_replay.py \\
  --gem5 build/Garnet_standalone/gem5.opt \\
  --source-trace traces/h100_8gpu_collectives.json \\
  --replay traces/h100_8gpu_allreduce_replay.json \\
  --sim-cycles 100000000
python3 tests/gem5/lab4/run_tensor_allreduce_trace_replay.py \\
  --gem5 build/Garnet_standalone/gem5.opt --sim-cycles 100000000
python3 tests/gem5/lab4/run_tensor_allreduce_paired.py \\
  --gem5 build/Garnet_standalone/gem5.opt
python3 report/scripts/build_final_report_data.py
```

The trace-replay runners use the same static-oracle bypass settings as G9. The source tree retains an optional adaptive selector for exploratory runs, but no headline number in this report depends on it.

8 Conclusion

The project evaluates three complementary mechanisms with three matching forms of evidence. Tree multicast shares duplicate prefixes and removes 39.51% of internal-link flits on average, with the saving rising to 53.13% at fanout 16; throughput remains workload dependent because atomic fanout can block a whole tree. Express links help only when topology, wire model, and load align: the static distance-scaled diagonal oracle reaches 1.629× arithmetic-mean latency speedup, while stride links average 0.981×. Tensor all-reduce preserves the collective as 15 multi-flit requests and shortens the scaled H100 replay window by 4.728× without changing rank contributions or Router flits.

Together these results support a narrow but useful design principle for collective-aware NoCs: expose structure where it removes demonstrable network work, admit physical shortcuts only with cost and congestion checks, select the aggressive or conservative policy according to whether peak speedup or tail stability is the priority, and evaluate request-level streaming with a completion contract that makes its units explicit.